

## TRACK 02 · 多智能体旅游服务方案

团队内部开发参考文档

## orchestrator/ 代码说明

给第一次看这份代码的人快速建立地图用的开发文档。`main.py` 只是薄薄的入口胶水代码，真正的 5 个 Agent 各自一个文件放在 `agents/` 目录下，`persona / store / trip_plan` 三个模块已经被对应的 Agent 接起来了（具体接了什么见文末），只有 OTA/酒店 Agent 还是纯占位假数据。

设计背景/接口契约见 [agent-architecture.md](#)（为什么这么分 Agent、为什么中心化编排）和 [agent-interfaces.md](#)（字段契约、原本的 Dify 方案，后来改成直接写 Python）。

### 环境准备

```
pip install openai python-dotenv chromadb
```

在 `orchestrator/.env` 里写这几行（这个文件已经在 `.gitignore` 里，不会被提交）：

```
PARATERA_API_KEY=你的key
AMAP_KEY=你的高德Web服务API Key
AMAP_JS_KEY=你的高德Web端(JS API)Key
AMAP_JS_SECURITY_CODE=你的安全密钥
```

高德的 key **分三样东西，缺一个都不行**：

- `AMAP_KEY`：**Web服务** 类型，后端用（`map_tool.py` 查地理编码/路线）
- `AMAP_JS_KEY`：**Web端(JS API)** 类型，前端画地图用——注意这是另一个应用/另一个 key，不能跟上面那个通用
- `AMAP_JS_SECURITY_CODE`：安全密钥，高德 JS API 2.0 版本强制要求，在申请 `AMAP_JS_KEY` 的同一个应用页面能找到

都在 <https://lbs.amap.com> 控制台“创建应用”里申请（一个应用可以同时创建 Web服务 + Web端(JS API) 两个 key，安全密钥在创建 Web端(JS API) key 的时候会一起给）。个人开发者未认证 6000次/天，实名认证后 30万次/月，demo 阶段够用。

每个文件都能直接 `python orchestrator/<路径>.py` 单独跑（包括 `agents/` 里的 5 个文件），文件末尾的 `if __name__ == "__main__":` 都是自测代码，可以照着抄用法。

目录结构

```
orchestrator/
  llm_tool.py           # 通用 LLM 调用工具，跟"编排"本身无关，谁都能 import
  map_tool.py          # 通用地图工具（高德地图 API）：地理编码 + 路径规划
  agents/
    orchestrator_agent.py # 编排 Agent: new_shared_state + classify_intent + orchestrate
    content_agent.py      # 达人/内容 Agent
    route_agent.py        # 行程/路线 Agent
    ota_hotel_agent.py    # OTA/酒店 Agent（还是纯占位）
    exception_agent.py    # 异常应变 Agent
  widgets.py            # 展示插件：右侧聊天框里的富交互小组件，跟 Agent 逻辑解耦
  schedule_widgets.py    # 左侧日程面板展示组件（地图+连线、每日时间线、机票酒店面板），跟 widgets.
  standardize_hk_macau_data.py # 把组员整理的港澳达人数据 Excel 转成标准化 JSON（见文末专门一节）
  import_hk_macau_data.py # 把标准化 JSON 灌进 Chroma（人格向量怎么算在这一步）
  data_sources/hk_macau_posts.json # 标准化后的港澳达人数据库（229 条，本地人认证 + 小红书两个来源）
  main.py               # 薄入口：re-export 编排 Agent 的两个函数 + __main__ 完整 demo
  store.py / trip_plan.py / persona.py # 不变
  server.py / web/      # 不变，还是 import main，接口不受这次拆分影响
```

拆分原则：谁负责哪个 Agent 就改 agents/ 下自己那一个文件，不用碰 main.py 或别人的 Agent 文件，减少多人协作冲突。

llm\_tool.py -- 通用 LLM 工具

**职责：**跟"编排"这件事本身无关的基础设施，5 个 Agent 里任何一个要调用模型都用这个，不要各自再写一遍 client 初始化。

| 函数/常量                                | 作用                                                                       |
|--------------------------------------|--------------------------------------------------------------------------|
| call_llm(messages, model=MODEL_FULL) | 统一的模型调用入口                                                                |
| MODEL_FULL / MODEL_LIGHT             | DeepSeek-V4-Pro / DeepSeek-V4-Flash，两档模型对齐 agent-architecture.md 的模型档位设计 |

map\_tool.py -- 通用地图工具（高德地图）

**职责：**地理编码 + 路径规划，给行程/路线 Agent 算真实的距离/耗时/路线用，跟 llm\_tool.py 一样是通用基础设施。

| 函数                                                                      | 作用                                                                                                                                        |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>geocode(address, city=None)</code>                                | 地名/地址转经纬度，返回 <code>(lng, lat)</code>                                                                                                      |
| <code>route_between(place_a, place_b, mode="walking", city=None)</code> | 算两地之间的距离/耗时/路线， <code>mode</code> 传 <code>"walking"</code> 或 <code>"driving"</code> ，返回 <code>distance_m / duration_min / polyline</code> |

现状/限制：

- 只做了“两点之间”的查询，还没做“多点一次性规划最优顺序”（这个高德也有专门接口，以后要优化路线顺序时再接）
- `route_agent.py` 里用这个算相邻两站的真实交通方式/耗时（超过 2km 自动从步行切驾车），查询失败会优雅降级成“待定（地图查询失败：...）”文字，不会导致整个流程崩掉
- 到达/结束的具体钟点时间（`arrival_time / end_time`）还是占位——那需要“一天几点开始”+“每站玩多久”这类还没定义的调度逻辑，跟地图 API 是两回事
- 地图可视化（真的在网页上画一张地图+路线）还没做，`web/index.html` 目前只是文字列表展示行程，`polyline` 字段目前没被前端用上

agents/orchestrator\_agent.py -- 编排 Agent

**职责：**星型架构的中枢。接收用户消息 → 意图识别 → 决定调用哪几个子 Agent → 汇总结果 → 生成回复。对应 [agent-interfaces.md 第三节](#) 的节点拓扑，只是从 Dify 可视化节点换成了 Python 函数。模型档位用 `MODEL_FULL`（全系统推理最重）。

| 函数                                                                                   | 作用                                                                                                                                                                 |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>new_shared_state(user_id, scenario="vacation", onboarding_answers=None)</code> | 造一份共享状态： <code>persona</code> （真用 <code>persona.py</code> 的结构，传了 <code>onboarding_answers</code> 就走冷启动）+ <code>trip_plan</code> （真用 <code>trip_plan.py</code> 的结构） |
| <code>classify_intent(user_message)</code>                                           | 调模型判断这轮要触发 <code>content / route / booking / exception</code> 里的哪几个                                                                                                |
| <code>orchestrate(user_message, shared_state)</code>                                 | 主流程：意图识别 → 分发调用 4 个子 Agent（子 Agent 直接改 <code>shared_state</code> 里的 <code>trip_plan</code> ，不用再手动写回）→ 生成回复，返回 <code>(output, shared_state)</code>                  |

`orchestrate()` 返回的 `output` 里除了 `chat_reply / community_panel / map_panel`，还有一个 `widgets` 数组（见下面 `widgets.py` 一节）：`content` 意图命中时会调 `widgets.build_post_list_widget()` + `widgets.build_attraction_picker_widget()`，命中候选为空就是空数组。

agents/content\_agent.py -- 达人/内容 Agent

**职责：**两阶段检索第一阶段。模型档位 `MODEL_FULL`（UGC 语义提炼），目前还没实际调用 LLM（纯向量检索）。

`run(shared_state, location_hint, top_k=3)`：算 `persona` 向量 → `store.query_similar_posts()` 查候选。  
`location_hint` 留给“内容相关性排序”（两阶段检索第二阶段）用，那部分还没实现。

返回的 `recommendations` 每条现在除了原有字段，还带港澳数据库的字段：

`city / category / post_type / address / tags / verified_local`（杭州那批老 demo 数据没有这些，会是 `None` / 空列表 / `False`，不影响原有字段读取）。

---

## agents/route\_agent.py -- 行程/路线 Agent

**职责：**把地点写进 `trip_plan` 某一天的行程节点。模型档位 `MODEL_LIGHT`，目前还没接 LLM，纯结构化写入。

`run(shared_state, places, time_budget=None, city=None)`：所有节点目前都写进同一个占位日期（`_PLACEHOLDER_DAY = "day-1"`）。`arrival_transport` 已经接了 `map_tool.py`，是相邻两站之间真实算出来的交通方式/耗时/距离（超过 2km 自动从步行切驾车）；`arrival_time / end_time` 还是占位文字“待定”（原因见 `map_tool.py` 那节）。

---

## agents/ota\_hotel\_agent.py -- OTA/酒店 Agent

**职责：**查库存/比价。模型档位 `MODEL_LIGHT`。**还是纯占位假数据**，没有真实的比价/查库存来源可接——但字段已经补全到“用户选中后能直接落地”的程度：机票候选带 `flight_no / from_ / to / depart_time / arrive_time / status`，酒店候选带 `address / check_in / check_out`（`check_in / check_out` 会用 `date_range` 参数填，格式“开始日期~结束日期”，没传就给占位日期）。这样 `POST /widget-response`（见 `server.py` 一节）收到用户选中的候选后，能直接拿这些字段调 `trip_plan.add_flight() / add_hotel()`，不用现造字段。

`orchestrate()` 里 `booking` 意图命中时会把这里返回的 `candidates` 分别喂给 `widgets.build_flight_picker_widget() / widgets.build_hotel_picker_widget()`（按 `candidates` 里的 `provider_type` 字段分流，见下面 `widgets.py` 一节），现在假数据一条机票一条酒店都有，两个 widget 都能出内容。

---

## agents/exception\_agent.py -- 异常应变 Agent

**职责：**按地点名字匹配 `trip_plan` 里的行程节点，命中就删掉并记一条天气异常。模型档位架构文档里写的是“中等模型”，目前只有 `MODEL_FULL / MODEL_LIGHT` 两档，先待定。

`run(shared_state, event_type, event_detail=None)`：`event_detail` 目前是纯子串匹配（整句用户消息去匹配地点名字），很粗糙，真实版本应该先做实体识别。

## widgets.py -- 展示插件（右侧聊天框富交互组件）

**职责：** 右侧聊天框里除了纯文字回复之外的富交互小组件（帖子列表、景点勾选等），跟 Agent/编排逻辑解耦——组件只负责"从真实候选集里挑/怎么展示"，不重新跑检索或编内容。

**设计原则：**

- 候选集永远来自真实代码算出来的数据（目前是 `content_agent.run()` 的 `recommendations`，已经是 `store.query_similar_posts()` 按人格相似度排过序的真实结果），不允许凭空捏造。
- 要用 LLM 做"选择/排序"时，LLM 只能从候选集里选 ID，不允许自己编内容；`_llm_select()` 会校验 LLM 返回的 ID 是否都在候选集里，筛掉任何编造的 ID（防幻觉），LLM 调用/解析失败时优雅降级成"按候选集原有顺序截取前 N 个"，不影响整体流程。
- 每个 widget 统一格式 `{"widget": "<type>", "data": {...}}`，前端按 `widget` 类型分发渲染，渲染逻辑完全不用管数据是怎么算出来的。
- 模型档位：`_llm_select()` 内部用 `llm_tool.MODEL_LIGHT`（选择/排序是结构化任务，不需要强模型）。

| 函数                                                                                                      | 作用                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>build_post_list_widget(candidates, top_k=3, use_llm_rerank=False)</code>                          | 帖子展示插件：默认直接截取候选集前 <code>top_k</code> 个（候选集已经是真实排序过的，不用额外调 LLM）；传 <code>use_llm_rerank=True</code> 走 LLM 二次筛选                                                                                                                                                                                         |
| <code>build_attraction_picker_widget(candidates, max_select=3, pool_size=6, use_llm_rerank=True)</code> | 景点选择插件：从候选里选出 <code>pool_size</code> 个放进"可选池"返回给前端，用户从池子里最多勾 <code>max_select</code> 个；默认走 LLM 精选候选池（LLM 只能选真实 ID）                                                                                                                                                                                   |
| <code>build_flight_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>     | 机票选择插件：跟 <code>build_attraction_picker_widget</code> 同一套"可选池 + 前端勾选 + 结果回传"契约，只是机票通常只订一个，默认 <code>max_select=1</code> ；从 <code>candidates</code> 里只挑 <code>provider_type == "flight"</code> 的，走 LLM 精选出 <code>pool_size</code> 个放进候选池；筛出来是空的就返回 <code>None</code> （调用方按 <code>None</code> 跳过，不展示空插件） |
| <code>build_hotel_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>      | 酒店选择插件：跟 <code>build_flight_picker_widget</code> 同一个模式，只挑 <code>provider_type == "hotel"</code> 的；同样可能返回 <code>None</code>                                                                                                                                                                           |

**依赖：**

- `content_agent.py` 返回的 `recommendations` 每条都带 `post_id` 字段（`store.query_similar_posts()` 本来就返回这个字段，之前 `content_agent.py` 组装返回值时漏传了，现已补上），`build_post_list_widget` / `build_attraction_picker_widget` 靠这个字段做 ID 校验。
- `build_flight_picker_widget` / `build_hotel_picker_widget` 的候选（`ota_hotel_agent.run()` 的 `candidates`）没有独立 id 字段，改用 `name` 字段当 `_llm_select` 的校验 key——跟 `post_id` 是同样的作用，只

是换了个字段名。

**四个 widget 分两种交互模式：** `post_list` 是纯展示（截取直接列出来）；  
`attraction_picker / flight_picker / hotel_picker` 是"可选池 + 前端勾选 + 结果回传"—— `options + max_select` 是统一契约，机票/酒店默认 `max_select=1`（通常只订一个），景点默认 `max_select=3`（可以多选几个）。

**现状：**四个 widget 都已经真正接进 `web/index.html` 渲染，`POST /widget-response` 接口也做完了（见 `server.py` 一节）——右侧聊天框里能看到真实的卡片/勾选交互，勾完点"确认选择"会真的把机票/酒店写进 `trip_plan`、把景点排进路线，不再是"传了但没人用"的状态。

**待接事项：**

- 讨论过的其他 widget 想法还没做：反馈评分插件、异常变更确认插件、人格问卷引导插件

schedule\_widgets.py -- 左侧日程面板展示组件

**职责：**服务对象是左侧实时日程面板（`server.py` 的 `GET /trip`，`web/index.html` 目前的文字列表），跟右侧聊天框的 `widgets.py` 是同一个"数据/展示分离"思路——这里只算"给前端画图/排版用的数据"，不产出任何 HTML/画布逻辑，真正怎么画留给前端。模型档位：不需要 LLM，纯确定性计算（按天/类型分配颜色 + 查真实地理数据）。三个 widget：

| 函数                                                           | 作用                                                                                                                                                                                                                                 |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>build_trip_map_widget(trip_plan_obj, city=None)</code> | 地图组件：把每天的景点/餐饮节点标进地图（同一天同一个颜色，颜色按"第几天"从固定调色板分配，超过 8 天循环复用），并把当天节点依次连成一条 <b>真实路线</b> （查真实路况，不是直线连线，颜色跟当天 markers 一致）；机票起降机场、酒店地址也各标一个点，用固定颜色（不参与按天上色，因为它们不属于"某一天"）                                                                  |
| <code>build_day_timeline_widget(trip_plan_obj)</code>        | 每日行程时间线：把每天的节点按时间顺序整理成竖排列表，按天分模块，模块颜色跟 <code>build_trip_map_widget</code> 里同一天的颜色对齐，方便地图和列表对照着看                                                                                                                                    |
| <code>build_booking_panel_widget(trip_plan_obj)</code>       | 机票/酒店面板：原样呈现 <code>trip_plan</code> 里已经落地确认的 <code>flights / hotels</code> 记录——注意这跟 <code>widgets.py</code> 的 <code>build_flight_picker_widget / build_hotel_picker_widget</code> 不是一回事，那两个是"聊天框里给用户挑的候选推荐"，这里是"已经确认、要显示在左侧行程里的记录" |

`build_trip_map_widget` 返回结构：

```
{
  "widget": "trip_map",
  "data": {
    "markers": [
      {
        "day": 1,
        "color": "#f96",
        "place": "Beijing",
        "node_id": 1,
        "type": "airport",
        "lng": 116.407,
        "lat": 39.904,
        ...
      },
      {
        "day": 1,
        "color": "#f96",
        "place": "Beijing",
        "node_id": 2,
        "type": "meal",
        "lng": 116.407,
        "lat": 39.904,
        ...
      },
      {
        "day": 1,
        "color": "#f96",
        "place": "Beijing",
        "node_id": 3,
        "type": "attraction",
        "lng": 116.407,
        "lat": 39.904,
        ...
      },
      ...
    ]
  }
}
```

```

"routes": [{"day":, "color":, "coordinates": [[lng,lat], ...]}],
"days_legend": [{"date":, "color":}],
"geocode_failures": [{"place":, "reason":}],
"route_failures": [{"day":, "from":, "to":, "reason":}],
}}

```

- 景点/餐饮 marker 的 `day / color` 按天分配；机票（`type: "airport"`，额外带 `role: "depart"|"arrive"`、`flight_no`、`status`）和酒店（`type: "hotel"`，额外带 `name / check_in / check_out`）的 `day` 固定是 `None`，颜色分别固定为深灰蓝/棕色（`_AIRPORT_COLOR / _HOTEL_COLOR`），不占用按天调色板
- `routes` 里的坐标是把 `map_tool.route_between()` 返回的多段 `polyline` 展平成一条连续的 `[lng, lat]` 序列，直接给前端画线；查询失败的那一段跳过并记进 `route_failures`，不影响其他天/其他段
- **字段约定：**`flights` 的 `from_ / to`、`hotels` 的 `address` 建议存能被地理编码识别的地名/地址（比如“杭州萧山国际机场”），不建议只存三字码（比如“HGH”）——高德地理编码认不出机场三字码，会直接进 `geocode_failures`

**现状：**三个 widget 已经接进 `server.py` 的 `GET /trip`，`web/index.html` 用高德 JS API 真的把地图画出来了（见 `server.py + web/index.html` 一节）；已经用真实 `AMAP_KEY` 端到端验证过，标点/连线/机票酒店坐标都是真实经纬度。

### 现状/限制：

- 跟 `route_agent.py` 一样依赖 `AMAP_KEY`（后端查经纬度/路线用的那个，注意不是前端画图用的 `AMAP_JS_KEY`），没配置时 `markers / routes` 会是空的、`*_failures` 里全是查询失败记录（不会导致整体报错，自测+真实端到端联调都验证过这个降级路径）
- `route_agent.py` 里已经算过一次相邻站点的交通方式/耗时，这里为了拿到完整 `polyline` 又独立查了一次路线（避免“展示层”反过来依赖某个 Agent 的内部计算结果），会有一点重复的高德 API 调用，demo/比赛规模的免费额度足够用，不是问题
- `POST /widget-response` 确认机票/酒店后会真的调 `trip_plan.add_flight() / add_hotel()`，`build_booking_panel_widget` / 机票酒店 marker 现在跑起来是有真实数据的（端到端联调过：选中机票+酒店 → `/trip` 能看到确认记录）
- **机票起降机场地理编码不传 `city`：**`build_trip_map_widget(trip_plan_obj, city=...)` 的 `city` 参数只用于景点/餐饮/酒店这些“行程主城市内”的地点，机场坐标查询故意不传 `city` ——接真实 key 测试时踩到过坑：查“上海虹桥国际机场”时如果传了 `city="杭州"`，高德会把它强行匹配成杭州萧山机场附近的结果（两个机场坐标几乎重合），因为出发/到达机场天然可能分属两个不同城市，用行程主城市限定反而会查错

## main.py -- 入口脚本

**职责：**纯胶水代码，不包含任何 Agent 逻辑。`from agents.orchestrator_agent import new_shared_state, orchestrate` 之后直接 re-export，`server.py` 还是 `import main` 调这两个函数，完全不用改。`__main__` 里是完整的端到端 demo（推荐 → 排路线 → 异常应变 → 打印最终行程）。

## store.py -- 达人社区内容 + 历史记录

**职责：**达人社区的图文帖子存储 + 检索（两阶段检索第一阶段：按人格向量相似度筛选），以及行程反馈历史记录。

**存储技术：**Chroma 向量库（orchestrator/data/chroma/，本地文件持久化）存帖子；JSON Lines 文件（orchestrator/data/history.jsonl）存历史记录；图片本地落盘（orchestrator/data/images/<post\_id>/），帖子里只存相对路径。

| 函数                                           | 作用                                                                                 |
|----------------------------------------------|------------------------------------------------------------------------------------|
| save_image(post_id, filename, image_bytes)   | 写一张图，返回存进 content["images"] 的相对路径                                                  |
| resolve_image_path(相对路径)                     | 相对路径转本地绝对路径，读图用                                                                    |
| add_post(post_id, persona_vector, content)   | 存一条帖子。content 字段见下表                                                                |
| query_similar_posts(persona_vector, top_k=5) | 两阶段检索第一阶段：按人格向量相似度找候选帖子，返回时带 similarity_score；verified_local=True 的帖子会被优先加权（见下面说明） |
| delete_post(post_id)                         | 删帖                                                                                 |
| log_history(user_id, trip_id, record)        | 追加一条行程反馈记录                                                                         |
| get_history(user_id=None)                    | 读历史记录，不传 user_id 读全部                                                               |

content 字段（add\_post 的第三个参数）：

| 字段                                                                 | 说明                                                                      |
|--------------------------------------------------------------------|-------------------------------------------------------------------------|
| place / time_slot / avg_cost / rating / avoid_tips / verified_trip | 结构化字段，对齐 agent-interfaces.md 达人 Agent 输出契约                              |
| caption                                                            | 帖子正文文字，可选                                                               |
| images                                                             | 图片相对路径列表，可选，由 save_image() 生成                                           |
| tags                                                               | 标签列表，可选，跟 images 一样序列化存（tags_json），查询时还原回列表                             |
| city / category / post_type / address / verified_local             | 港澳达人数据库用的字段（见 import_hk_macau_data.py 一节），verified_local=True 表示本地人认证来源 |

**"本地人认证"优先加权是怎么做的：**query\_similar\_posts() 先按 top\_k \* 3 多捞一些候选（不是只捞 top\_k 个），把 verified\_local=True 的帖子相似度乘一个固定系数 \_VERIFIED\_LOCAL\_BOOST = 1.15，再按加权后的分数

重新排序截到 `top_k` 。这个"先多捞再重排"是必须的——如果直接对 Chroma 已经按原始距离截好的 `top_k` 结果加权，加权只能在这几条里面换个顺序，换不进被漏掉的本地人帖子，加权就没意义了。

待定/限制：

- 内容相关性排序（两阶段检索第二阶段）没实现，留给调用方在 `query_similar_posts()` 的候选集里自己排
- 图片存本地文件系统，部署到 ModelScope Studio 时要换成对象存储（OSS/CDN），`images` 字段到时候存 URL 而不是本地路径，上层调用方式不变
- `_VERIFIED_LOCAL_BOOST` 系数（1.15）是拍的，没有做过效果对比调优，觉得加权效果不明显/太强可以直接改这个常量

standardize\_hk\_macau\_data.py + import\_hk\_macau\_data.py -- 港澳达人数据库

**背景：**组员手动整理了一份港澳（澳门/香港）达人推荐/避坑数据，存在仓库根目录的 `数据库原始.xlsx` 里，来源分两种：

- **本地人认证**（Excel 里的"本地人澳门"/"本地人香港" sheet）：饮食/景点/Tips 分类推荐 + 单独一张避坑表
- **小红书**（"小红书澳门"/"小红书香港" sheet）：褒贬都有的普通帖子，不算本地人认证来源

这两种来源对应 `store.py` 里新加的 `verified_local` 字段——本地人认证的帖子在检索时会被优先加权（见 `store.py` 一节），小红书来源不加权。这个字段是原始数据自带的真实来源标记（组员整理的时候已经分好表了），不是靠算法从内容"判断"出来的。

**两步流程**（分开是为了标准化出错时不用重新灌一次向量库）：

| 脚本                                        | 作用                                                                                                                   |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>standardize_hk_macau_data.py</code> | 读 <code>数据库原始.xlsx</code> ，标准化字段 + 编号，写出 <code>data_sources/hk_macau_posts.json</code> 。不碰 Chroma                    |
| <code>import_hk_macau_data.py</code>      | 读标准化 JSON，用 <code>persona.infer_post_persona_vector()</code> 给每条帖子算一个 人格向量，调 <code>store.add_post()</code> 灌进 Chroma |

**编号规则**（城市前缀 + 类型字母 + 三位数字，类型内连续编号，不区分本地人/小红书来源）：

|      | 澳门·推荐                        | 澳门·避坑                  | 香港·推荐                        | 香港·避坑                  |
|------|------------------------------|------------------------|------------------------------|------------------------|
| 编号格式 | <code>MF/MS/MT</code> + 三位数字 | <code>MN</code> + 三位数字 | <code>HF/HS/HT</code> + 三位数字 | <code>HN</code> + 三位数字 |
| 示例   | MF001, MS001, MT001          | MN001, MN002           | HF001, HS001, HT001          | HN001, HN002           |
| 说明   | 饮食/景点/Tips，正常分类              | 全部统一放这里，不分饮食/景点        | 同左                           | 同左                     |

解读：`M` = Macau，`H` = Hong Kong（城市前缀）；`F` = Food，`S` = Sight，`T` = Tips（推荐分类）；`N` = No（避坑专用，统一放一起不细分）。

**原始表格解析上的坑：**Excel 里不同分类小节的列不是对齐的（比如"本地人香港"的 Tips 小节没有单独的"名称"列，内容直接写在名称那一格；"本地人香港"饮食小节比"本地人澳门"多一列"推荐人"），`standardize_hk_macau_data.py` 按每个小节自己的表头动态取列，不能假设固定列位置——这个坑是解析的时候踩出来的，写死列位置会导致数据错位。

**当前数据量**（229 条，跑 `python standardize_hk_macau_data.py` 会打印这个分布）：

| 前缀 | 条数              | 前缀 | 条数             |
|----|-----------------|----|----------------|
| MF | 50（本地人30+小红书20） | HF | 19（本地人9+小红书10） |
| MS | 50              | HS | 20             |
| MT | 50              | HT | 20             |
| MN | 10              | HN | 10             |

**已知数据缺口：**香港饮食类本地人认证只有 9 条，不是目标的 10 条——原始表格里就是这样，组员应该是漏填了一条，不是解析脚本的 bug，不影响其他数据使用。

**怎么重新跑：**

```
python orchestrator/standardize_hk_macau_data.py # 改了 Excel 或标准化逻辑之后跑
python orchestrator/import_hk_macau_data.py      # 改了人格反推逻辑，或者清空过 orchestrator/data/chrc
```

`import_hk_macau_data.py` 用的是 `upsert`（`store.add_post()` 内部调的），同一个 id 重复导入会覆盖不会重复，可以放心重跑。

**待接事项：**

- 避坑类帖子（MN / HN）反推出来的人格向量是中性默认值（见 `persona.py` 一节的已知局限），检索排名上不占优势也不吃亏，更合理的做法（不管人格匹配度、只要地点/类型对上就该出现）属于两阶段检索第二阶段，还没实现
- `verified_local` 优先加权的系数（1.15）没有做过 A/B 效果对比，是拍的经验值
- `content_agent.py` 的两阶段检索第二阶段接上之后，`category`（饮食/景点/Tips）字段可以用来做更精细的"用户问吃的就优先出饮食类"这种过滤，现在还没用上

## trip\_plan.py -- 旅游计划（行程/机票/酒店/天气异常）

**职责：**一次完整旅行的所有结构化信息，由多个 Agent 共同维护，最终汇总用于呈现。

**顶层结构：**

```
trip_plan = {
    "trip_id": "...",
    "flights": [...], # OTA/酒店 Agent 维护
```

```
"hotels": [...],          # OTA/酒店 Agent 维护
"weather_alerts": [...], # 异常应变 Agent 维护
"days": {日期: day_plan},# 行程 Agent 生成, 异常应变 Agent 增删改
}
```

**景点通勤+餐饮的核心设计:** 每天一个"链表", 但不是指针式链表, 而是**节点字典 + id 引用** ( day\_plan["nodes"] 是 {node\_id: node} , 每个节点有 next\_id 指向下一个节点的 id, day\_plan["head\_id"] 是链表头) 。这样设计是因为:

- 异常应变 Agent 改行程时只需要动被影响的那一个节点、接一下前后 next\_id , 不用重发整天的数组
- 本质是个普通 dict, 直接能存 JSON/传 HTTP, 不需要额外的链表↔数组转换逻辑

节点字段: type ( "attraction" | "meal" ) 、 place 、 arrival\_transport 、 arrival\_time 、 end\_time 、 next\_id ; meal 类型节点多一个 meal\_type (早餐/午餐/晚餐/夜宵) 。

| 函数                                                                                                           | 作用                                                                           |
|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| new_trip_plan(trip_id)                                                                                       | 造一个空旅游计划                                                                     |
| add_flight / add_hotel / add_weather_alert                                                                   | 往对应列表追加一条记录                                                                  |
| get_or_create_day(trip_plan, date)                                                                           | 拿到某天的 day_plan , 不存在就新建                                                      |
| add_stop(day_plan, node_id, type_, place, arrival_transport, arrival_time, end_time, after_id=None, **extra) | 插入一个节点, after_id=None 插到最前面, 否则插到该节点后面                                       |
| remove_stop(day_plan, node_id)                                                                               | 删除节点, 自动重连前后节点 (异常应变 Agent 常用, 比如景点临时关闭)                                     |
| patch_stop(day_plan, node_id, **fields)                                                                      | 只改某个节点的部分字段, 不动链表结构                                                          |
| day_stops(day_plan)                                                                                          | 把链表还原成有序数组, 渲染/地图面板用                                                         |
| render(trip_plan)                                                                                            | 汇总成最终展示结构<br>( flights / hotels / weather_alerts / days , 每天的 stops 已经是有序数组) |

**现状:** 已经接进 main.py 的 shared\_state["trip\_plan"] , agent\_route 会往里面加节点、 agent\_exception 会删节点/记天气异常。目前所有节点都写进同一个占位日期 "day-1" ( main.py 里的 \_PLACEHOLDER\_DAY ) , 还没做真正的多日期规划。

## persona.py -- 人格变量

**职责:** 定义"旅游人格"的具体维度结构, 供达人/行程/OTA Agent 读取, 反馈闭环用来校准。

**维度设计**（能量化的维度都用了 0~1 连续分数，不是纯标签）：

| 维度             | 归属                          | 类型                                         | 说明                                                                            |
|----------------|-----------------------------|--------------------------------------------|-------------------------------------------------------------------------------|
| taste          | stable_traits<br>(跨场景稳定)    | 固定词表 multi-hot                             | 菜系标签，词表见 _TASTE_VOCAB                                                         |
| novelty_score  | stable_traits               | float 0~1                                  | 0=稳妥大众，1=小众冒险                                                                 |
| pace_score     | scenario_traits<br>(按场景分开存) | float 0~1                                  | 0=完全休闲，1=完全特种兵                                                                |
| budget_score   | scenario_traits             | float 0~1                                  | 0=穷游，1=公务舱/总统套房/租车这种享受型消费； <b>硬过滤字段，不进相似度向量</b> ，用 budget_filter_ok() 按数值区间过滤 |
| social_mode    | scenario_traits             | 分类<br>( solo / couple / family / friends ) | 并列模式非程度光谱，向量里按 one-hot 编码                                                     |
| interest_theme | scenario_traits             | 固定词表 multi-hot                             | 词表见 _INTEREST_VOCAB                                                           |

scenario\_traits 按"本次旅行模式" ( business\_trip / vacation / family / solo\_adventure ...) 分开存，同一个人不同场景下这些值可以完全不同。

| 函数                                                                                        | 作用                                                                                    |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| new_persona(user_id)                                                                      | 造一份空人格                                                                                |
| bootstrap_from_onboarding(user_id, scenario, answers)                                     | 冷启动：从一份问卷答案组出初始 persona                                                               |
| set_scenario_traits(persona, scenario, **fields) / get_scenario_traits(persona, scenario) | 读写某场景的特质，写入时会校验词表 + clamp 数值到 0~1                                                     |
| update_stable_traits(persona, **fields)                                                   | 读写跨场景稳定特质，同样有校验                                                                       |
| apply_feedback(persona, scenario, dimension, new_value, reason="")                        | 按维度校准，写入 + 留痕 ( feedback_log )。"该改成什么值"由调用方决定，这里只负责写入                                 |
| budget_filter_ok(a_score, b_score, max_diff=0.25)                                         | 判断两个 budget_score 是否够接近，硬过滤专用，独立于相似度向量                                                |
| compute_persona_vector(persona, scenario)                                                 | 拼出向量：[novelty_score, taste multi-hot, pace_score, social_mode one-hot, interest_theme |

| 函数                                                                                        | 作用                                                                         |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
|                                                                                           | multi-hot]，固定 21 维（词表大小变了维度也会变，加了"葡国菜"后从 20 变 21）                          |
| <code>infer_post_persona_vector(scenario, tags=None, avg_cost=None, category=None)</code> | 港澳达人数据库导入专用：这些帖子是组员手动整理的真实内容，没有"发帖人自己的问卷答案"，用标签/人均/分类反推一个大致合理的人格向量，见下面单独说明 |

**infer\_post\_persona\_vector()** 是怎么反推的（启发式估计，不是精确画像）：

- `_TAG_TO_TASTE / _TAG_TO_INTEREST` 两个关键词映射表，对标签文本做子串匹配，匹配上就计入对应的 `taste / interest_theme`；匹配不上的标签（比如"三代传承" "400年历史"这种描述性但不构成人格维度信号的）直接丢弃，不强行凑
- `category`（饮食/景点）也会补一条弱信号：饮食→"美食探店"，景点→"人文历史"
- `avg_cost` 线性换算成 `budget_score`（0~250 元映射到 0~1，250 元封顶），换算公式很粗糙，不是精确定价模型
- 标签里出现"小众/冷门/隐世"这类词 → `novelty_score` 偏高（0.75）；出现"网红/热门/必打卡" → 偏低（0.3）；都没有就是中性 0.5
- `social_mode` 故意不猜——单条帖子看不出"适合独行/家庭/朋友"，留空对应 `compute_persona_vector()` 里那段 one-hot 全 0，这是合法的"未指定"编码
- **已知局限**：避坑（避雷理由）类帖子在标准化数据里没有 `tags / avg_cost / category`，反推出来的人格向量是中性默认值（`novelty=0.5, budget=0.5`，无 `taste/interest`），跟任何用户人格的相似度都不会特别高也不会特别低——这类帖子更适合"不管人格匹配度、只要地点/类型对上就该出现"，属于两阶段检索第二阶段（内容相关性排序）该管的事，那部分还没实现

待定/限制：

- `compute_persona_vector()` 是数值特征拼接，不是真实语义 embedding，以后接真实 embedding 模型可以整体替换掉函数体，调用方（`store.py`）接口不用变
- 改 `_TASTE_VOCAB / _INTEREST_VOCAB` 词表会改变向量维度，如果本地 Chroma 已经有存量数据，改词表前要清掉 `orchestrator/data/chroma` 重新灌数据，不然新旧向量维度对不上（这个坑已经踩过一次，这次加"葡国菜"也重新灌了一次数据）
- 从一条用户反馈文本判断"该把哪个维度调成什么值"的算法没实现，留给编排/达人 Agent 决定

server.py + web/index.html -- 呈现层（网页）

**职责**：把 `main.py` 的编排 Agent、`widgets.py`、`schedule_widgets.py` 包成 HTTP 接口 + 真实渲染，配一个两栏网页——左边实时日程（地图+每日时间线+机票酒店面板），右边聊天框（文字回复+四个插件卡片）。这是目前唯一"从对话到落地预订"整条链路能真的跑通的地方。

运行方式：

```
pip install flask python-dotenv
python orchestrator/server.py
```

然后浏览器打开 `http://127.0.0.1:5000`。地图能不能真的画出来，取决于 `.env` 里三个高德 key 有没有配全（见“环境准备”一节）；没配的话页面照样能跑，只是地图是空的、控制台会有一条警告。

**接口**（都是同一个 Flask app 提供，同源不用处理 CORS）：

| 接口                    | 作用                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /                 | 返回 <code>web/index.html</code> ，服务端会把文件里的 <code>__AMAP_JS_KEY__</code> / <code>__AMAP_JS_SECURITY_CODE__</code> 占位符换成 <code>.env</code> 里的真实值再返回（这两个 key 不写进 git 里的 html 文件，跟其他密钥一样只活在 <code>.env</code> ）                                                                                                                                                            |
| GET /trip             | 返回 <code>trip_plan.render()</code> 的行程 + 额外三个字段 <code>trip_map</code> / <code>day_timeline</code> / <code>booking_panel</code> （分别是 <code>schedule_widgets.py</code> 那三个函数的输出），左侧日程面板用这个渲染                                                                                                                                                                            |
| POST /chat            | body 传 <code>{"message": "..."} </code> ，内部调 <code>main.orchestrate()</code> ，返回编排 Agent 的输出（ <code>chat_reply</code> / <code>community_panel</code> / <code>map_panel</code> / <code>widgets</code> ）                                                                                                                                                                |
| POST /widget-response | body 传 <code>{"widget": "attraction_picker" "flight_picker" "hotel_picker", "selected": [...]}</code> ， <code>selected</code> 是前端从对应 widget 的 <code>options</code> 里原样拿到的候选对象（不是前端自己编的字段，跟 <code>widgets.py</code> 里“LLM 只能选真实 ID”是同一个防幻觉思路）。景点选中后调 <code>route_agent.run()</code> 排进行程；机票/酒店选中后调 <code>trip_plan.add_flight()</code> / <code>add_hotel()</code> 确认预订 |

**前端逻辑**（`web/index.html`，原生 HTML/CSS/JS + 高德地图 JS SDK，没引入前端框架）：

- 发消息 → 调 `/chat` → 显示文字回复 → 把返回的 `widgets` 数组按类型分发渲染（`post_list` 是横滑卡片；`attraction_picker` / `flight_picker` / `hotel_picker` 共用同一套“复选框 + 确认按钮”组件，`max_select` 决定最多能勾几个）→ 再调一次 `/trip` 刷新左侧日程
- 三个“选择型”widget 点“确认选择”之后：把勾中的候选对象原样 `POST /widget-response`，成功后往聊天记录里加一条系统提示（“已确认：xxx”），再刷新一次左侧日程
- 左侧地图用 `AMap.Map` 初始化一次（页面加载时），之后每次 `/trip` 刷新只 `map.clearMap()` 重画：景点/餐饮用 `AMap.CircleMarker` 按天上色，机票/酒店 marker 固定深灰蓝/棕色（颜色跟 `schedule_widgets.py` 里的常量对应，改了那边记得这边也要改），路线用 `AMap.Polyline` 按天上色，标记点击会弹 `AMap.InfoWindow` 显示地点名
- 每日行程时间线现在读 `trip.day_timeline`（带颜色）而不是原始的 `trip.days`，模块颜色跟地图上同一天的颜色对得上

**现状/限制：**

- 全局只有一个进程内共享状态（`server.py` 里的 `_state`），是单会话 demo，没有登录/多用户/并发处理，仅供本地演示，不要直接这样部署到公网
- 已经端到端联调过完整链路：“推荐+排路线+查机票酒店”发过去 → 右侧出现 `flight_picker` / `hotel_picker` → 模拟前端 `POST /widget-response` 选中一个航班一个酒店 → 再查 `/trip` 能看到 `flights` / `hotels` 里多了确认记录（这一步是直接打 API 验证的，没配 `AMAP_JS_KEY` 时浏览器里的地图看不到，但数据链路是通的）

- 社区图文面板（三面板设计里"逛社区帖子墙"那块，跟聊天框里的 `post_list` 不是一回事）还没做进网页
- 地图是"重新全量清空再画"，不是增量更新，行程节点很多的时候会有一点点闪烁，demo 规模不明显

## 待接事项

`main.py / server.py` 已经把所有 Agent + 两套 widgets 串成一条完整链路：跑 `python orchestrator/main.py` 能看到一次"推荐→规划路线→异常应变→重新渲染行程"的端到端流程；跑 `python orchestrator/server.py` 打开网页，能看到"聊天推荐→勾选景点/机票/酒店→确认→左侧地图/时间线/机票酒店面板更新"这条更完整的闭环真的在跑（已端到端联调验证过）。还剩这几处没做，都不阻塞基本演示：

1. `ota_hotel_agent` 还是纯占位假数据（固定一条机票一条酒店），没有真实比价/查库存来源可接
2. `route_agent` 目前所有节点都写进同一个占位日期 `_PLACEHOLDER_DAY = "day-1"`，没有真正的多日期规划——这意味着"按天上色"这个视觉设计在真实多日行程里还体现不出来（永远只有一种颜色）；交通方式/耗时已经接了高德地图 API 真实计算，但到达/结束的具体钟点时间还是占位文字 "待定"
3. `exception_agent` 用整句用户消息去子串匹配地点名字，很粗糙（比如"西湖"两个字出现在消息里就命中），真实版本应该先做实体识别抽出具体地点
4. 两阶段检索第二阶段（候选集内部按内容相关性排序）还没实现，`content_agent` 里 `location_hint` 参数目前没用上
5. 反馈闭环（用户点评行程 → 校准 persona）还没接：`store.log_history()` 记录和 `persona.apply_feedback()` 校准都写好了，但没人在 `orchestrate()` 里调用它们
6. `map_tool.py` 目前只查"两点之间"，没做"多点最优顺序"规划（比如给定 5 个景点，没有算出最优游览顺序，只是按用户/LLM 给的顺序排）
7. 三个高德 key 已经申请配置好并真实验证过（西湖→灵隐寺路线、机场/酒店坐标都是真实高德数据）
8. 讨论过的其他 widget 想法还没做：反馈评分插件、异常变更确认插件、人格问卷引导插件
9. 单会话全局 state（`server.py` 里的 `_state`），没有登录/多用户/并发处理，真要多人同时用需要重新设计状态管理
10. 港澳达人数据库（229 条，见 `standardize_hk_macau_data.py` 一节）已经标准化+导入 Chroma，`verified_local` 优先加权也接上了，但避坑类帖子的人格向量是中性默认值、`content_agent` 的 `category` 字段还没用来做精细过滤，这两个都属于两阶段检索第二阶段（第4条）的范畴